Employee Management System

A simple desktop application built with Python, Tkinter and CSV files.

Submitted by

Nihar Bute
Roll Number: 09

Nandini More
Roll Number: 06

Subject

Python Programming

Project type

Second-year engineering Python mini-project

The project demonstrates classes, objects, functions, loops, conditions, file handling and exception handling through five employee record operations.

Project overview

1. Introduction

The Employee Management System is a basic desktop application for storing and maintaining employee records. Each record contains an employee ID, name, department and salary. A Tkinter form accepts input and a table displays the saved records. CSV storage keeps the information available after the application closes.

2. Problem Statement

Keeping employee details in separate handwritten notes makes it difficult to find, correct and remove records. A small, consistent application can organize these details and reduce repeated manual work. This project provides a simple solution suitable for learning Python programming.

3. Objectives

- Add an employee with a unique ID, name, department and salary.
- View all saved employees in a clear table.
- Search for an employee using the employee ID.
- Update an existing employee's name, department or salary.
- Delete a record after confirmation.
- Validate input and save changes in a CSV file for later use.

Scope

The application uses Python 3 and the standard library only. It is intended for one user and one running instance. The employee ID acts as a unique, case-sensitive key. Clear resets the form without deleting stored records.

Design and Python concepts

4. Methodology

Start: Create an EmployeeApp object and build the form, buttons and table. Read employees.csv from the same folder as main.py. If the CSV does not exist, create it with the required headings.

Accept and validate input: Trim surrounding spaces and check that required fields are filled. Reject duplicate IDs. Convert salary to a finite, nonnegative number and format it to two decimal places.

Process an action: Read records into a list of dictionaries. Add appends a record; Search matches an ID; Update changes the matching record; Delete removes it after confirmation. View shows the complete list.

Save and refresh: Write changed records to a temporary CSV and replace the original file after writing succeeds. Refresh the table and show a status message. When the application restarts, load the saved file again.

5. Python Concepts Used

Classes and objects: EmployeeApp groups application data and behavior; app is its instance.

Functions: Methods such as read_form, load_records and run divide the work into readable parts.

Loops and conditions: Loops build fields and display rows; if/elif statements choose the requested action.

File handling: csv.DictReader and csv.DictWriter read and write records using with statements.

Exception handling: try/except catches invalid input and file errors and displays a readable message.

Lists and dictionaries: A dictionary represents one employee; a list stores all employees.

6. Source Code

main.py - complete source, part 1 of 4

```
"""Employee Management System - a simple Tkinter and CSV mini-project."""

import csv
import math
from pathlib import Path
import tkinter as tk
from tkinter import messagebox, ttk


DATA_FILE = Path(__file__).resolve().with_name("employees.csv")
FIELDS = ("Employee ID", "Name", "Department", "Salary")


class EmployeeApp:
    """Display the form and manage employee records in a CSV file."""

    def __init__(self, root, data_file=DATA_FILE):
        self.root = root
        self.data_file = Path(data_file)
        root.title("Employee Management System")
        root.geometry("900x560")
        root.minsize(760, 480)
        root.configure(padx=20, pady=16)
        ttk.Label(root, text="Employee Management System",
                  font=("Segoe UI", 20, "bold")).pack(pady=(0, 16))

        form = ttk.LabelFrame(root, text="Employee information", padding=12)
        form.pack(fill="x")
        self.inputs = {}
        for index, field in enumerate(FIELDS):
            row, column = divmod(index, 2)
            ttk.Label(form, text=field).grid(
                row=row, column=column * 2, padx=8, pady=8, sticky="w")
            value = tk.StringVar()
            ttk.Entry(form, textvariable=value, width=28).grid(
                row=row, column=column * 2 + 1, padx=8, pady=8, sticky="ew")
            self.inputs[field] = value
        form.columnconfigure(1, weight=1)
        form.columnconfigure(3, weight=1)

        buttons = ttk.Frame(root)
        buttons.pack(fill="x", pady=14)
        self.buttons = {}
        for action in ("Add", "View", "Search", "Update", "Delete", "Clear"):
            button = ttk.Button(buttons, text=action,
                                command=lambda name=action: self.run(name))
            button.pack(side="left", expand=True, fill="x", padx=3)
            self.buttons[action] = button

        footer = ttk.Frame(root)
        footer.pack(side="bottom", fill="x", pady=(10, 0))
        self.status = tk.StringVar(value="Ready")
```

6. Source Code

main.py - complete source, part 2 of 4

```
        ttk.Label(footer, textvariable=self.status).pack(anchor="w")
        ttk.Label(footer, text="Select a row to edit. Employee ID is the unique key."
                  ).pack(anchor="w", pady=(4, 0))

        table_frame = ttk.Frame(root)
        table_frame.pack(fill="both", expand=True)
        self.table = ttk.Treeview(table_frame, columns=FIELDS,
                                  show="headings", selectmode="browse")
        for field in FIELDS:
            self.table.heading(field, text=field)
            self.table.column(field, width=190, minwidth=100,
                              anchor="e" if field == "Salary" else "w")
        scroll = ttk.Scrollbar(table_frame, orient="vertical",
                               command=self.table.yview)
        self.table.configure(yscrollcommand=scroll.set)
        scroll.pack(side="right", fill="y")
        self.table.pack(side="left", fill="both", expand=True)
        self.table.bind("<<TreeviewSelect>>", self.select_record)

        self.run("View")

    def load_records(self):
        """Read the CSV; create a header-only file on the first run."""
        if not self.data_file.exists():
            self.save_records([])
        with self.data_file.open("r", newline="", encoding="utf-8") as file:
            reader = csv.DictReader(file)
            if reader.fieldnames != list(FIELDS):
                raise ValueError("CSV headings are invalid. Restore employees.csv.")
            records = list(reader)
        seen = set()
        for record in records:
            if None in record or any(not record.get(key) for key in FIELDS):
                raise ValueError("CSV contains an incomplete employee record.")
            if record["Employee ID"] in seen:
                raise ValueError("CSV contains duplicate employee IDs.")
            seen.add(record["Employee ID"])
        return records

    def save_records(self, records):
        """Write to a temporary file, then replace the previous CSV."""
        temporary = self.data_file.with_suffix(".tmp")
        with temporary.open("w", newline="", encoding="utf-8") as file:
            writer = csv.DictWriter(file, fieldnames=FIELDS)
            writer.writeheader()
            writer.writerows(records)
        temporary.replace(self.data_file)

    def read_form(self):
        record = {key: value.get().strip() for key, value in self.inputs.items()}
        if not all(record.values()):
            raise ValueError("Please fill in all four fields.")
```

6. Source Code

main.py - complete source, part 3 of 4

```
        try:
            salary = float(record["Salary"])
        except ValueError:
            raise ValueError("Salary must be a valid number.") from None
        if not math.isfinite(salary) or salary < 0:
            raise ValueError("Salary must be a finite number of zero or more.")
        record["Salary"] = f"{salary:.2f}"
        return record

    def display(self, records):
        for item in self.table.get_children():
            self.table.delete(item)
        for record in records:
            self.table.insert("", "end", values=[record[key] for key in FIELDS])

    def fill_form(self, record):
        for key in FIELDS:
            self.inputs[key].set(record[key])

    def clear_form(self):
        for value in self.inputs.values():
            value.set("")
        self.table.selection_remove(self.table.selection())

    def select_record(self, event=None):
        selected = self.table.selection()
        if selected:
            values = self.table.item(selected[0], "values")
            self.fill_form(dict(zip(FIELDS, values)))

    def run(self, action):
        """Handle the six buttons and show friendly errors."""
        try:
            if action == "Clear":
                self.clear_form()
                self.status.set("Input fields cleared.")
                return
            records = self.load_records()
            if action == "View":
                self.display(records)
                self.status.set(f"Showing all {len(records)} employee(s).")
                return

            employee_id = self.inputs["Employee ID"].get().strip()
            if not employee_id:
                raise ValueError("Please enter an employee ID.")
            employee = next((record for record in records
                             if record["Employee ID"] == employee_id), None)

            if action == "Add":
                if employee:
                    raise ValueError("This employee ID already exists.")
```

6. Source Code

main.py - complete source, part 4 of 4

```
                records.append(self.read_form())
            else:
                if employee is None:
                    raise ValueError("No employee found with this ID.")
                if action == "Search":
                    self.fill_form(employee)
                    self.display([employee])
                    self.status.set(f"Found employee {employee_id}.")
                    return
                if action == "Update":
                    employee.update(self.read_form())
                elif action == "Delete":
                    if not messagebox.askyesno(
                            "Confirm deletion", f"Delete employee {employee_id}?",
                            parent=self.root):
                        return
                    records.remove(employee)

            self.save_records(records)
            self.display(records)
            self.clear_form()
            self.status.set(f"{action} successful for employee {employee_id}.")
        except (OSError, UnicodeError, csv.Error, ValueError) as error:
            messagebox.showerror("Employee Management System", str(error),
                                 parent=self.root)
            self.status.set("Action failed. Please check the error message.")


if __name__ == "__main__":
    window = tk.Tk()
    app = EmployeeApp(window)
    window.mainloop()
```

7. Output

Actual screenshot of the running Tkinter application. The three employees are fictional demonstration records included in employees.csv.

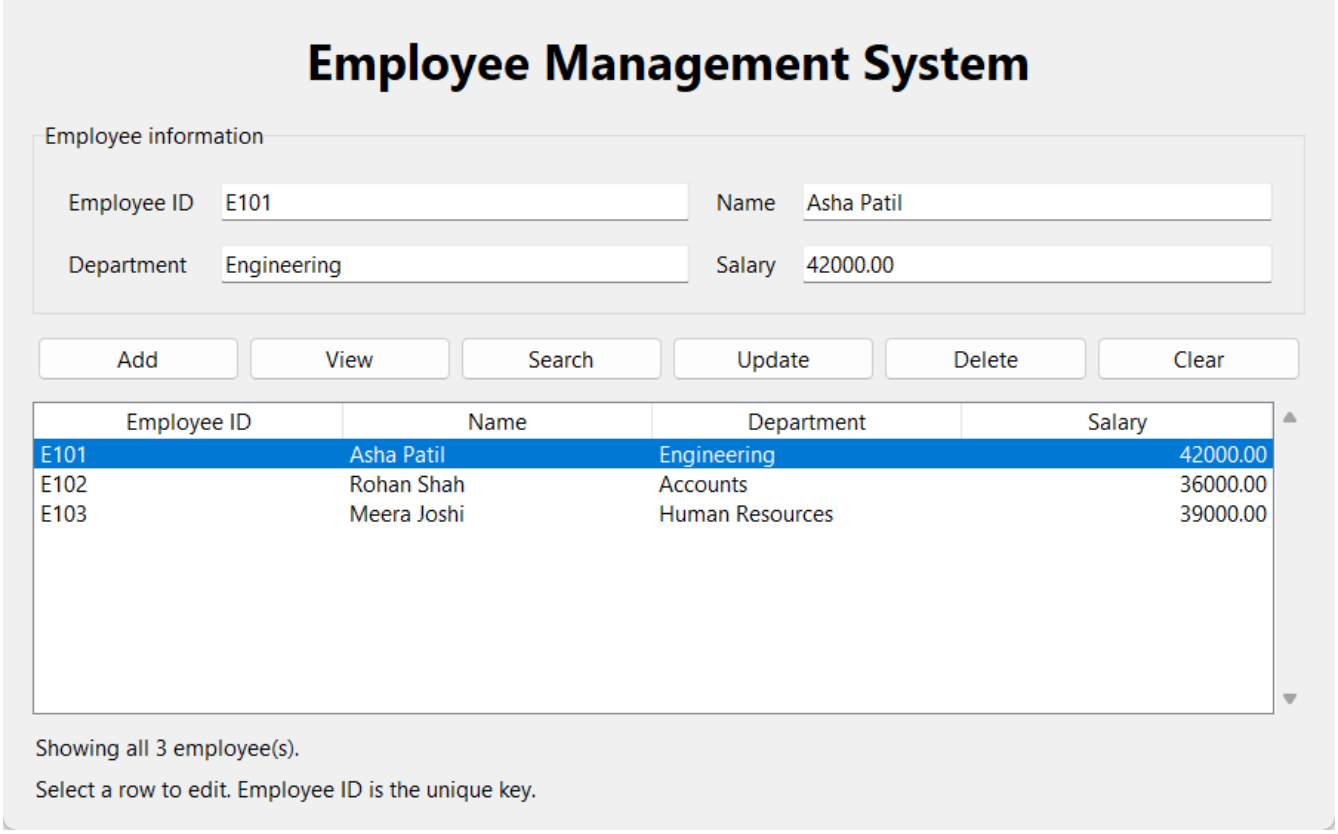

Figure 1: The application shows its input form, six buttons, employee table and status message. Selecting employee E101 fills the form for editing.

Running the project

Open a terminal in the project folder and run `python main.py`. On Windows, `py main.py` can also be used. Python must include Tkinter; no third-party application packages are required.

Using the form

For Add, complete all four fields. For Search or Delete, enter the employee ID. For Update, select or search for a record, keep its ID unchanged, edit the other values and click Update. Click View to restore all rows after a search.

Verification and conclusion

Testing performed

The application was executed on Windows with Python 3.12 and Tkinter 8.6. Checks invoked the real GUI buttons against a separate test CSV. The test script controlled dialog responses and then started a fresh Python process to verify persistence. The screenshot was captured from the live GUI.

Add, View and Search: Passed. Records were saved, shown and found by ID; names containing commas were preserved.

Update and Delete: Passed. Changed fields were saved. Cancelling deletion kept the row; confirming removed it.

Restart persistence: Passed. A new process loaded the updated record and the deleted employee stayed absent.

Validation: Passed for duplicate IDs, blank input, missing records and invalid, negative or non-finite salary.

File errors: Malformed headings displayed an error. A simulated write failure left the original CSV intact.

8. Conclusion

The project provides the five required employee management operations in a simple graphical application. It demonstrates how basic Python concepts can work together with CSV storage to preserve records between sessions. The design remains small enough to explain during a practical examination.

9. GitHub Repository Link

https://github.com/niiharbute/Employee-Management-System

The public repository contains the application source, sample CSV, README, actual output screenshot and this report.